

Segment3D: Learning Fine-Grained Class-Agnostic 3D Segmentation without Manual Labels

Rui Huang¹, Songyou Peng², Ayça Takmaz², Federico Tombari³,
Marc Pollefeys^{2,4}, Shiji Song¹, Gao Huang^{1,*}, and Francis Engelmann^{2,3}

¹Tsinghua University ²ETH Zurich
³Google ⁴Microsoft *Corr. Author
<https://segment3d.github.io>



Class-agnostic Segmentation



Mask2Former



SAM - Segment Anything Model

Context

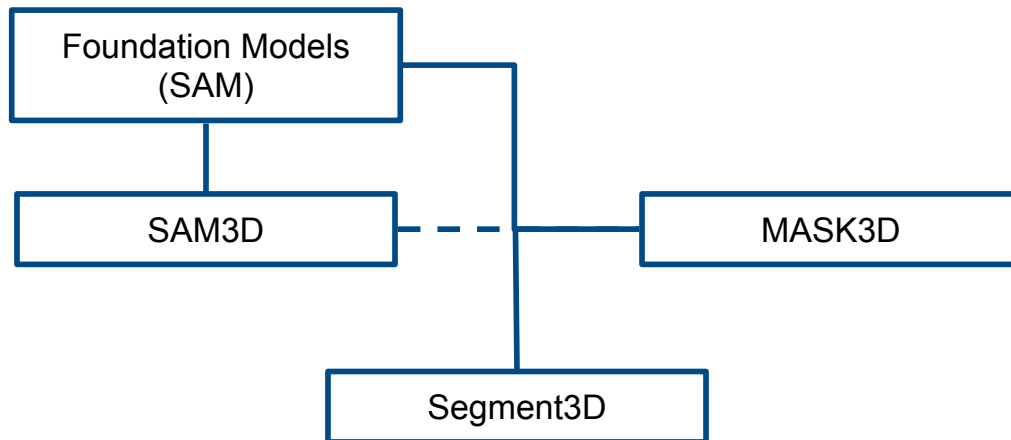
Foundation Models
(SAM)

Context

Foundation Models
(SAM)

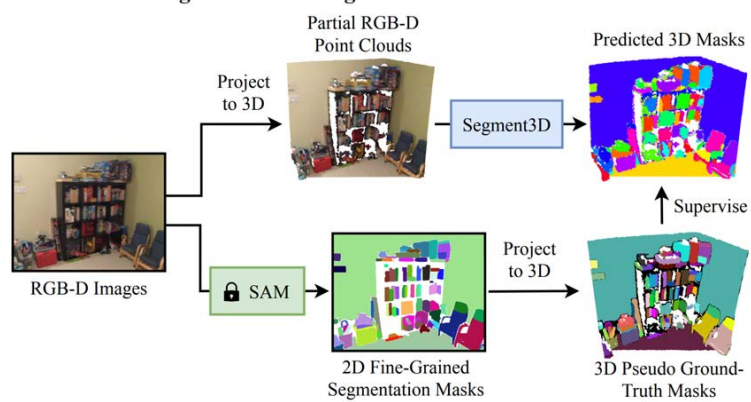
SAM3D

Context

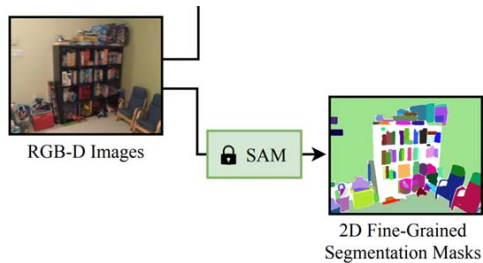


Train - Stage 1

Stage1: Pre-Training on Partial Point Clouds

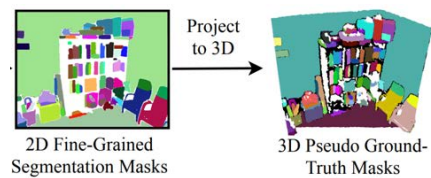


Train - Stage 1 - SAM



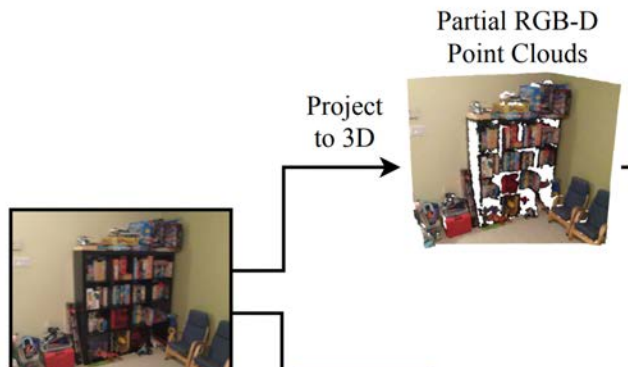
- Receives xy + RGB-D input
- Uses the SAM model to predict 2D masks

Train - Stage 1 - Project to 3D



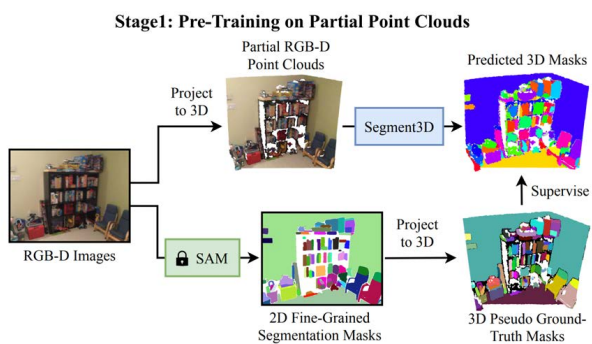
- After predicting the pixel-wise masks, utilize the depth data to reconstruct a 3D point cloud.
- Camera intrinsic parameters.

Train - Stage 1 - Project to 3D



- We also project the original RGB-D data into a 3D point cloud

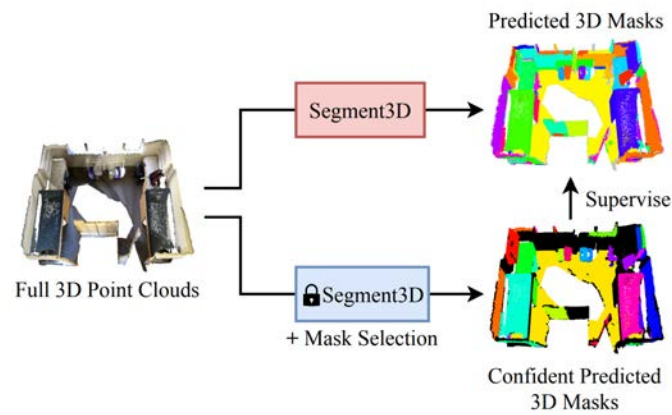
Train - Stage 1 - Segment3D



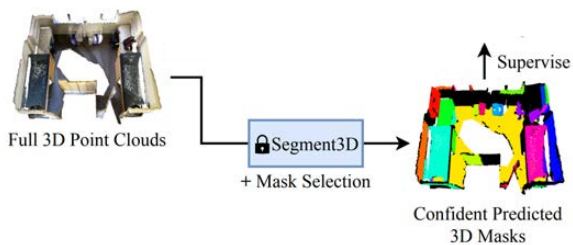
- We use the projected mask predicted by SAM as the "ground truth" and train with it

Train - Stage 2

Stage2: Fine-Tuning on Full Point Clouds



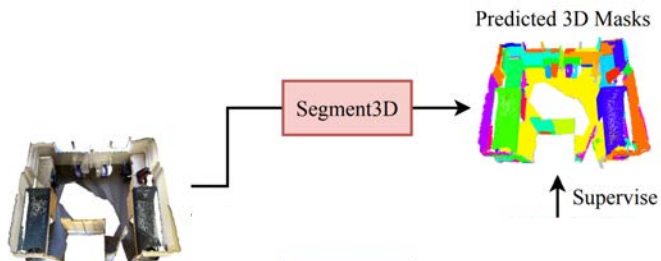
Train - Stage 2



- Once the model has learned to generate 3D point cloud masks, it uses the trained model to generate new masks on a point cloud and selects only those with the highest confidence

Train - Stage 2

Stage2: Fine-Tuning on Full Point Clouds



- We fine-tune the model using these predicted labels as pseudo-labels

Segment3D Architecture

- “We use a model inspired by Mask3D”
- “Sparse convolutional backbone derived from MinkowskiUNet”
- “Transformer decoder”

Implementation

- For CUDA 11.3 / PyTorch 1.12 (It won't work in the 5090)
- So, changes had to be made, to made

Component	Version	Verified
torch + torchvision	2.7.1+cu128 / 0.22.1	matmul on sm_128 ✓
torch-scatter	2.1.2+pt27cu128	GPU scatter ✓
MinkowskiEngine	0.5.4 (+ custom patch)	GPU sparse conv/pool ✓
pointnet2	local	GPU FPS ✓
pytorch-lightning	1.9.5	imports, gpus= still valid (no code edits) ✓
cuML	26.02	GPU KMeans/DBSCAN ✓
detectron2	from main	point_sample + comm ✓
repo deps + demo.py	-	full import chain ✓

- Other minor changes to the code

Rum demo



Rum demo Radix Data

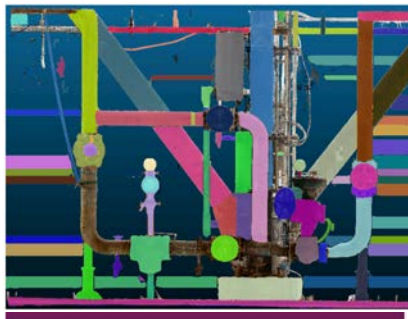


Applying SAM – CC screenshot

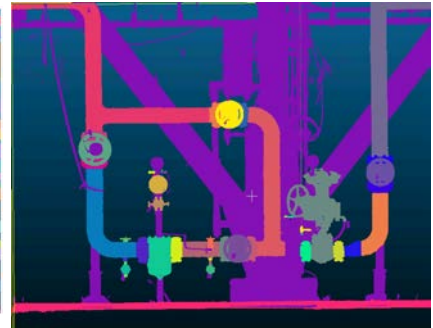
Cloud screenshot



SAM's segmentation



Original cloud index

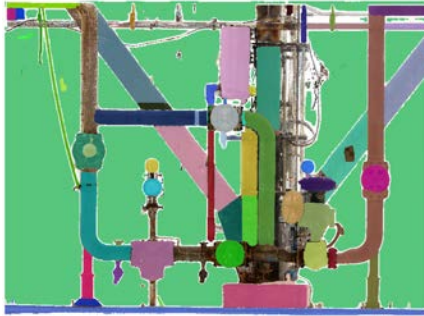


Applying SAM – white background

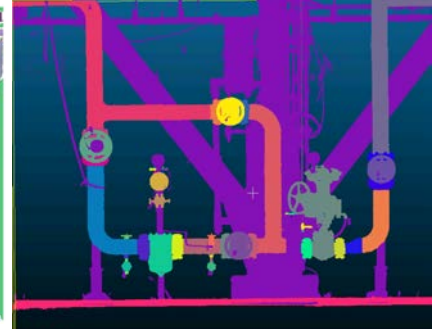
Cloud screenshot



SAM's segmentation

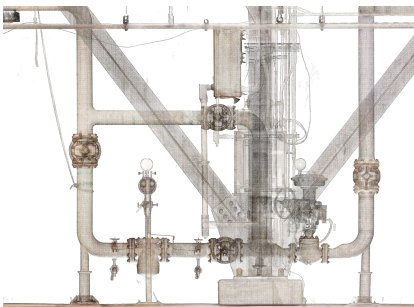


Original cloud index

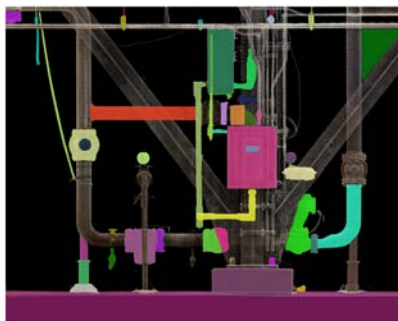


Applying SAM – CC automatic projection

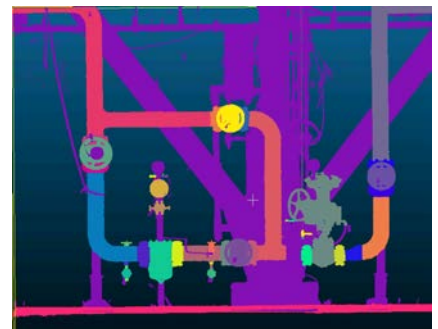
Cloud projection



SAM's segmentation

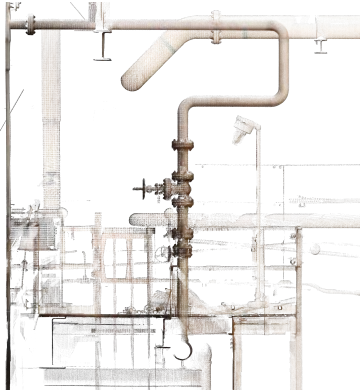


Original cloud index



Applying SAM – Another example

Cloud projection



SAM's segmentation



Original cloud index



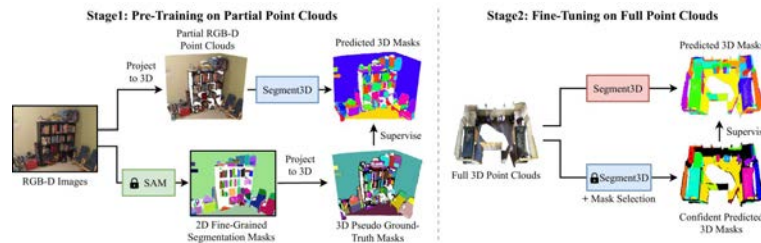
The review

- The paper is evaluated along five axes:
summary, strengths, weaknesses, rating, suggestions.
- Guiding principles followed:
 - weigh **novelty and impact**, not only state-of-the-art margins;
 - be **specific** about prior art;
 - keep feedback **constructive**.

Summary

- **Dataset:** Point clouds vs RGB-D.

- **Method:**



- **Results:** beats supervised Mask3d; Generalizes to not trained datasets + open-vocabulary.



"A black eraser"

"Kettle handle"

"Copier control screen"

Strengths

- **Innovation (9/10):** Tackle the problem of small datasets for points clouds by adapting a larger dataset with RGB-D images and project it in (partial) 3d point clouds. It mainly get together many new approaches and classical algorithms to achieve better results;
- **Results (9/10):** Construct a model robust to different kinds of scenes, able to keep a good performance even in datasets with no training or training without labels and also in Open-vocabulary results;
- **Writing (7/10):** It is a solid paper cites well all models used and all datasets trained and evaluated;
- **Reproducibility (6/10):** The code is publicly available in [github](https://github.com).

Qualitative Results in ScanNet++



Table 1: Segmentation Scores on ScanNet++ [58]. The metric is average precision (AP) on the validation split. We report scores with and without post-processing (more details in Sec. 4.1). For reference, we show results on Mask3D trained on manual ScanNet200 labels.

Model	Avg. Inference Time/s	without post-processing			with post-processing		
		AP	AP ₅₀	AP ₂₅	AP	AP ₅₀	AP ₂₅
Mask3D [46]	0.7	8.7	15.5	27.2	14.3	21.3	29.9
SAM3D [57]	386.7	3.9	9.3	22.1	8.4	16.1	30.0
Felzenszwalb <i>et al.</i> [20]	12.6	5.8	11.6	27.2	—	—	—
Segment3D (Ours)	0.7	13.0	23.8	38.3	20.2	30.9	42.7

Table 2: Segmentation Scores on ScanNet200 [13, 44]. The evaluation metric is average precision on the val split.

Model	AP	AP ₅₀
Mask3D [46]	34.1	43.1
Felzenszwalb <i>et al.</i> [20]	6.1	12.1
UnScene3D [45]	15.9	32.2
SAM3D [57]	19.0	32.5
Segment3D (Ours)	27.7	39.8

Table 3: Segmentation Scores on nuScenes [4]. The evaluation metric is average precision on the val split.

Model	GT Labels	AP ₅₀	AP ₂₅
<i>zero-shot test</i>			
Mask3D	×	15.3	25.5
Segment3D	×	37.8	48.0
<i>fine-tuned on nuScenes</i>			
Mask3D	✓	46.5	57.8
Segment3D	×	42.5	55.5

Open-vocabulary Results

Table 7: Results with Additional Training Data. The performance is further boosted with more training data.

Pre-Training Data	AP	AP ₅₀
ScanNet	13.0	23.8
ScanNet, ScanNet++	17.1 (+4.1)	26.7 (+2.9)

Table 8: Open-Set 3D Scene Understanding. The evaluation metric is average precision with an IoU threshold of 50%.

Segmentor	ScanNet++	Replica
Mask3D [46]	15.0	18.0
Segment3D (Ours)	18.5 (+3.5)	18.4 (+0.4)



"a black eraser"

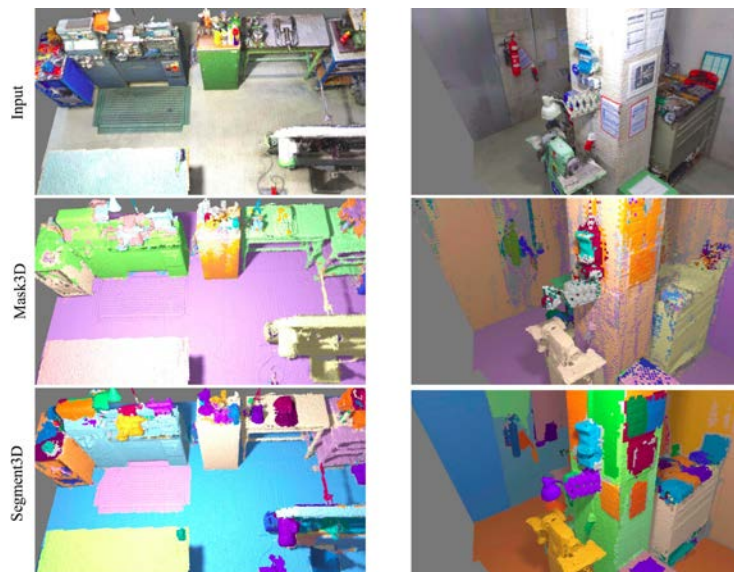


"paper on the laptop"

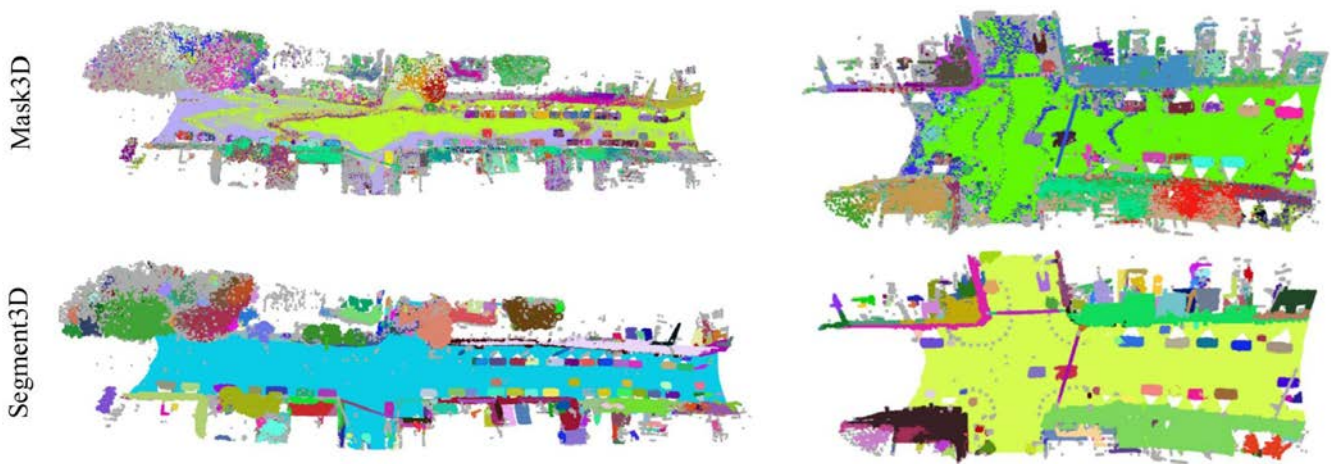


"scissors"

In-the-wild scenes



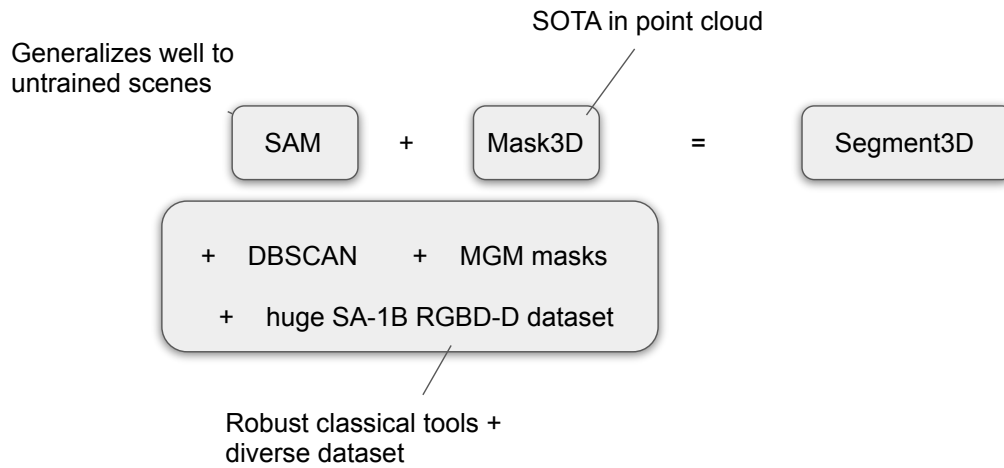
Paris-Lille



Weakness

- **“Without manual labels” – relocation, not elimination.** No 3D annotations are used, but all supervision is inherit from SAM, itself trained on SA-1B (~1B human-guided masks). The labeling cost is moved upstream, not removed.
- **Reproducibility friction.** Released code doesn't build with current CUDA/PyTorch; dependencies are pinned to outdated versions and require manual reworking to run.
- **Curated results, no failure analysis.** Only favorable qualitative examples are shown. Expected weak spots are never illustrated – hard to judge real robustness.

Final considerations



Rating & justification

Recommendation:  Accept.

Innovation	9
Results	9
Writing	7
Reproducibility	6

Future Work and Research Suggestions



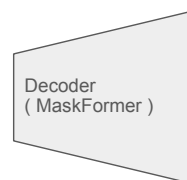
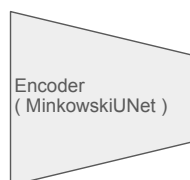
Safe routes vs Not-so-safe routes vs Wild Territory

Future Work and Research Suggestions



Safe routes:

Backbones

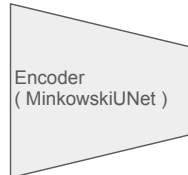


- └ PTv3 - 2024
- └ Swin3d (shifted window attention) - 2023
- └ SPVCNN (point-voxel convolution) - 2020

Future Work and Research Suggestions

Safe routes:

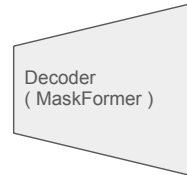
Backbones



- PTv3 - 2024
- Swin3d (shifted window attention) - 2023
- SPVCNN (point-voxel convolution) - 2020

Not-so-safe routes:

Decoders and more



- Mask2Former* (cross-attention -> masked attention) - 2022
- OneFormer* (text-conditioned decoder) - 2023

Future Work and Research Suggestions

Wild territory:

Gaussian Splatting

SegSplat <https://arxiv.org/pdf/2511.18386>

Z-Order Transformer <https://arxiv.org/pdf/2605.13465>

SemGS <https://arxiv.org/pdf/2603.02548>

Gaussian Splatting Transformers <https://arxiv.org/pdf/2409.04196>

TranSplatting <https://arxiv.org/pdf/2408.13770>

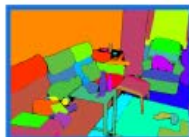
Gaussian Grouping: Segment and Edit Anything in 3D Scenes <https://arxiv.org/pdf/2312.00732>

Gaga: Group Any Gaussians via 3D-aware Memory Bank <https://arxiv.org/pdf/2404.07977>

Future Work and Research Suggestions

Wild territory: Gaussian Splatting

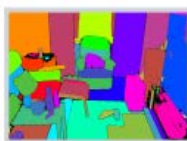
Gaga:
Group Any Gaussians via 3D-aware Memory Bank



Multi-view Consistent Seg.



Group Any Gaussians via 3D-Aware Memory Bank



Rendered Seg.



Feature Field



Ac

Future Work and Research Suggestions

Wild territory: Gaussian Splatting

SemGS:
Feed-Forward Semantic 3D Gaussian
Splatting from Sparse Views for
Generalizable Scene Understanding

